

Orquestador concurrente de modelos

Objetivo: simular llamadas a tres modelos de IA con latencias diferentes, ejecutarlas concurrentemente con `asyncio.gather`, manejar un límite total de 2 segundos y limitar a 2 las llamadas activas al programar 10 simulaciones. Se utiliza únicamente la biblioteca estándar, sin llamadas a APIs reales.

Criterios de aceptación

Criterio	Evidencia	Estado
No usar <code>time.sleep</code> (bloqueante).	Las tres corrutinas utilizan <code>await asyncio.sleep(latency)</code> .	✓ Cumplido
Las tareas inician casi al mismo tiempo.	Los logs incluyen milisegundos. La primera ejecución inicia las tres llamadas concurrentemente.	✓ Cumplido
Manejar correctamente <code>TimeoutError</code> .	Cada ejecución captura <code>TimeoutError</code> , muestra un mensaje y permite continuar o finalizar normalmente.	✓ Cumplido

Ejemplo real de ejecución

Primera ejecución: tres modelos simultáneamente
[21:47:01.006] GPT-4: inicio de la llamada
[21:47:01.007] Claude-3: inicio de la llamada
[21:47:01.008] Local-llama: inicio de la llamada
Error: las tres llamadas superaron el límite total de 2 segundos.

Segunda ejecución: diez llamadas, como máximo dos a la vez
[21:47:03.009] GPT-4: inicio de la llamada
[21:47:03.010] Claude-3: inicio de la llamada
[21:47:04.314] Local-llama: inicio de la llamada
[21:47:04.723] GPT-4: inicio de la llamada
Error: las diez llamadas superaron el límite total de 2 segundos.

Verificación: ejecución con Python 3.13, finalizada con código de salida 0. Las latencias aleatorias entre 0,5 y 2,5 segundos pueden producir resultados distintos en cada ejecución. El `timeout` es un comportamiento esperado y manejado; cancela las llamadas pendientes. En la segunda ejecución comienzan solo dos llamadas y las siguientes acceden a medida que se libera un lugar. Con este rango, las 10 llamadas no pueden completar dentro de 2 segundos.

Código completo - main.py

```
"""Ejercicio de orquestación concurrente de modelos."""

import asyncio
import random
from collections.abc import Awaitable, Callable
from datetime import datetime

async def gpt_4_call() -> str:
    """Simular una llamada a GPT-4 y devolver una respuesta de texto."""
    hora_inicio = datetime.now().strftime("%H:%M:%S.%f")[:-3]
    print(f"[{hora_inicio}] GPT-4: inicio de la llamada")
    latency = random.uniform(0.5, 2.5)
    await asyncio.sleep(latency)
    return "Respuesta simulada de GPT-4"

async def claude_3_call() -> str:
    """Simular una llamada a Claude 3 y devolver una respuesta de texto."""
    hora_inicio = datetime.now().strftime("%H:%M:%S.%f")[:-3]
    print(f"[{hora_inicio}] Claude-3: inicio de la llamada")
    latency = random.uniform(0.5, 2.5)
    await asyncio.sleep(latency)
    return "Respuesta simulada de Claude 3"

async def local_llama_call() -> str:
    """Simular una llamada a Llama local y devolver una respuesta de texto."""
    hora_inicio = datetime.now().strftime("%H:%M:%S.%f")[:-3]
    print(f"[{hora_inicio}] Local-llama: inicio de la llamada")
    latency = random.uniform(0.5, 2.5)
    await asyncio.sleep(latency)
    return "Respuesta simulada de Llama local"
```

Código completo - continuación

```
async def main() -> None:
    """Coordinar las llamadas, limitar la concurrencia y manejar el timeout."""
    print("Primera ejecución: tres modelos simultáneamente")
    try:
        async with asyncio.timeout(2):
            resultados = await asyncio.gather(
                gpt_4_call(),
                claude_3_call(),
                local_llama_call(),
            )
            for resultado in resultados:
                print(resultado)
    except TimeoutError:
        print("Error: las tres llamadas superaron el límite total de 2 segundos.")

semaforo = asyncio.Semaphore(2)

async def ejecutar_con_limite(
    modelo: Callable[[], Awaitable[str]],
) -> str:
    """Ejecutar una llamada cuando haya un lugar disponible."""
    async with semaforo:
        return await modelo()

print("\nSegunda ejecución: diez llamadas, como máximo dos a la vez")
modelos = [gpt_4_call, claude_3_call, local_llama_call]
llamadas = []
for indice in range(10):
    modelo = modelos[indice % len(modelos)]
    llamadas.append(ejecutar_con_limite(modelo))

try:
    async with asyncio.timeout(2):
        resultados = await asyncio.gather(*llamadas)
        for resultado in resultados:
            print(resultado)
except TimeoutError:
    print("Error: las diez llamadas superaron el límite total de 2 segundos.")

if __name__ == "__main__":
    asyncio.run(main())
```